

Name : Yamini Aruna M

Accord info matrix

Datazoic task

SYSTEM DESIGN DOCUMENT: SCALABLE AGENTIC ARCHITECTURE

Executive Summary & Problem Breakdown

Core Problem (The Scaling Bottleneck)

When an agent is connected to 50+, 100+, or thousands of APIs, passing every schema directly into the prompt degrades LLM performance:

- **Attention Dilution:** Transformer attention disperses across large contexts, causing the agent to miss constraints or select wrong tools.
- **Hallucinations & Collisions:** Endpoints with overlapping names lead to mixed parameters and invalid headers.
- **High Latency & Token Cost:** Inlining hundreds of schemas inflates token usage on every conversational turn.

Proposed Solution (Decoupled Discovery & Execution)

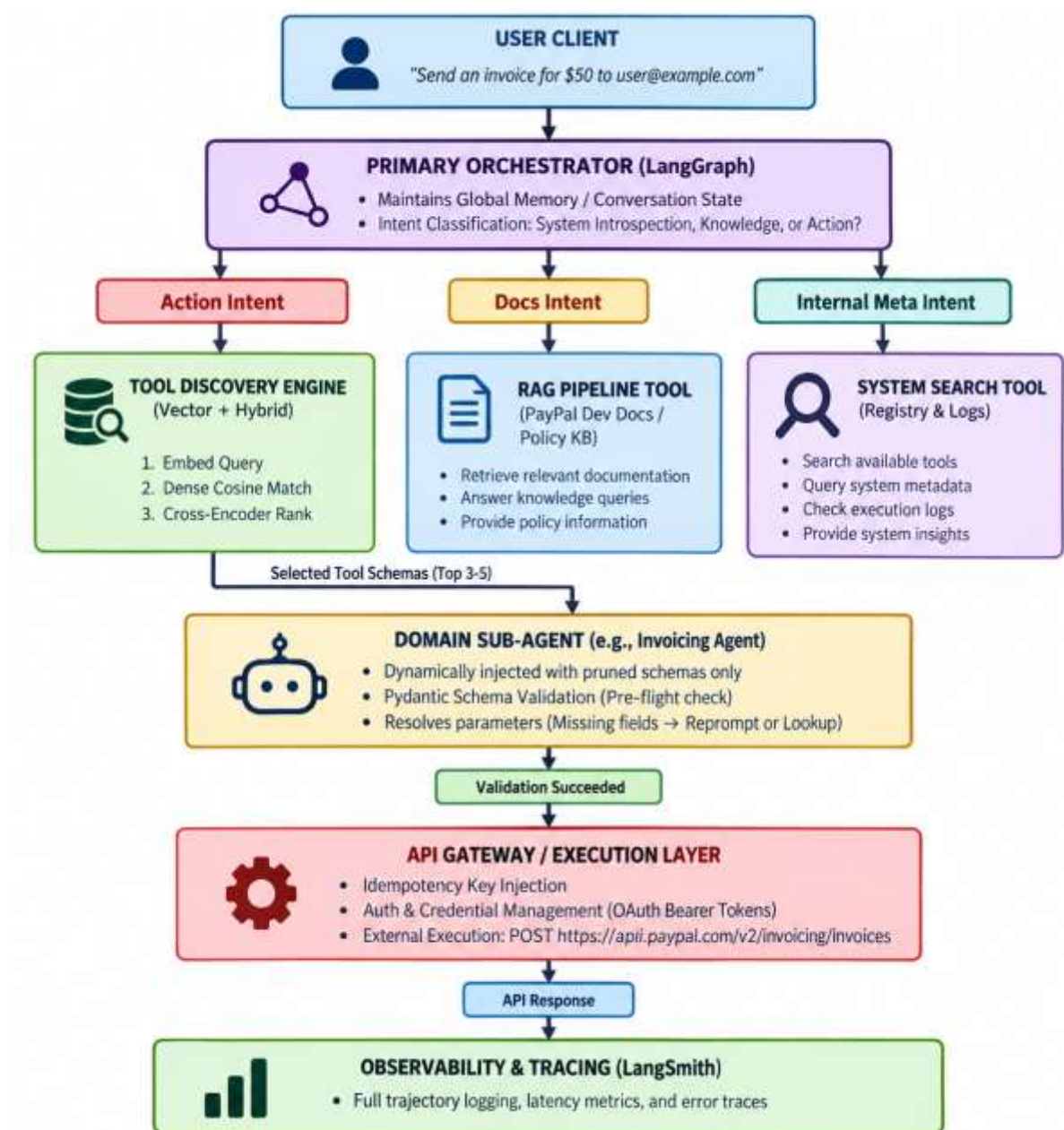
Instead of exposing the entire tool catalog to the LLM, decouple tool discovery from tool execution:

- **Central Tool Registry:** Stores metadata and parameters for all tools in a vector database.
- **Discovery Engine:** Uses hybrid semantic search to dynamically retrieve only the top 3–5 schemas relevant to the user’s request.
- **Sub-Agent Execution:** Sandboxed workers receive only the retrieved schemas, validate inputs, execute the APIs, and manage multi-step workflows.

End-to-End System Architecture

The architecture separates responsibilities into four layers:

1. **User Interaction & Orchestrator:** Manages dialog context, extracts intent, and governs agent routing.
2. **Tool Discovery Engine (RAG for APIs):** Performs semantic search over an indexed tool registry to surface relevant schemas.
3. **Execution Sub-Agents:** Sandboxed workers receiving only the pruned schemas, armed with structured validation.
4. **State, Verification & Observability:** Enforces determinism, tracks state history, handles errors, and records traces via LangSmith.



Core Engine Mechanics

A. Dynamic Tool Discovery Engine

Instead of exposing 500 API schemas to the orchestrator, tool specifications are indexed inside a vector database (e.g., Qdrant).

Each tool entry is indexed with rich semantic metadata:

{

"tool_id": "paypal_invoicing_send",

"category": "payments.invoicing",

"name": "Send Invoice",

```
"description": "Sends an invoice to a customer given a valid invoice ID. Updates the status to SENT.",
```

```
"parameters": ["invoice_id", "notify_customer", "subject"],
```

```
"sample_queries": [
```

```
  "send invoice 102",
```

```
  "email bill to customer",
```

```
  "dispatch invoice"
```

```
]
```

```
}
```

Step 1 (Dense Retrieval): The user's query is embedded, and cosine similarity surfaces the top 15 candidate tools.

Step 2 (Cross-Encoder Re-ranking): A light cross-encoder model scores candidate relevance against conversational context, picking the top 3–5 tools.

Step 3 (Dynamic Schema Injection): Only the selected schemas are loaded into the sub-agent prompt, preventing context pollution.

B. Specialized Sub-Agents

Tools are divided into operational domains (e.g., Invoicing, Dispute Management, Reporting, Transfers). The Orchestrator delegates tasks to targeted Sub-Agents, keeping their focus narrow.

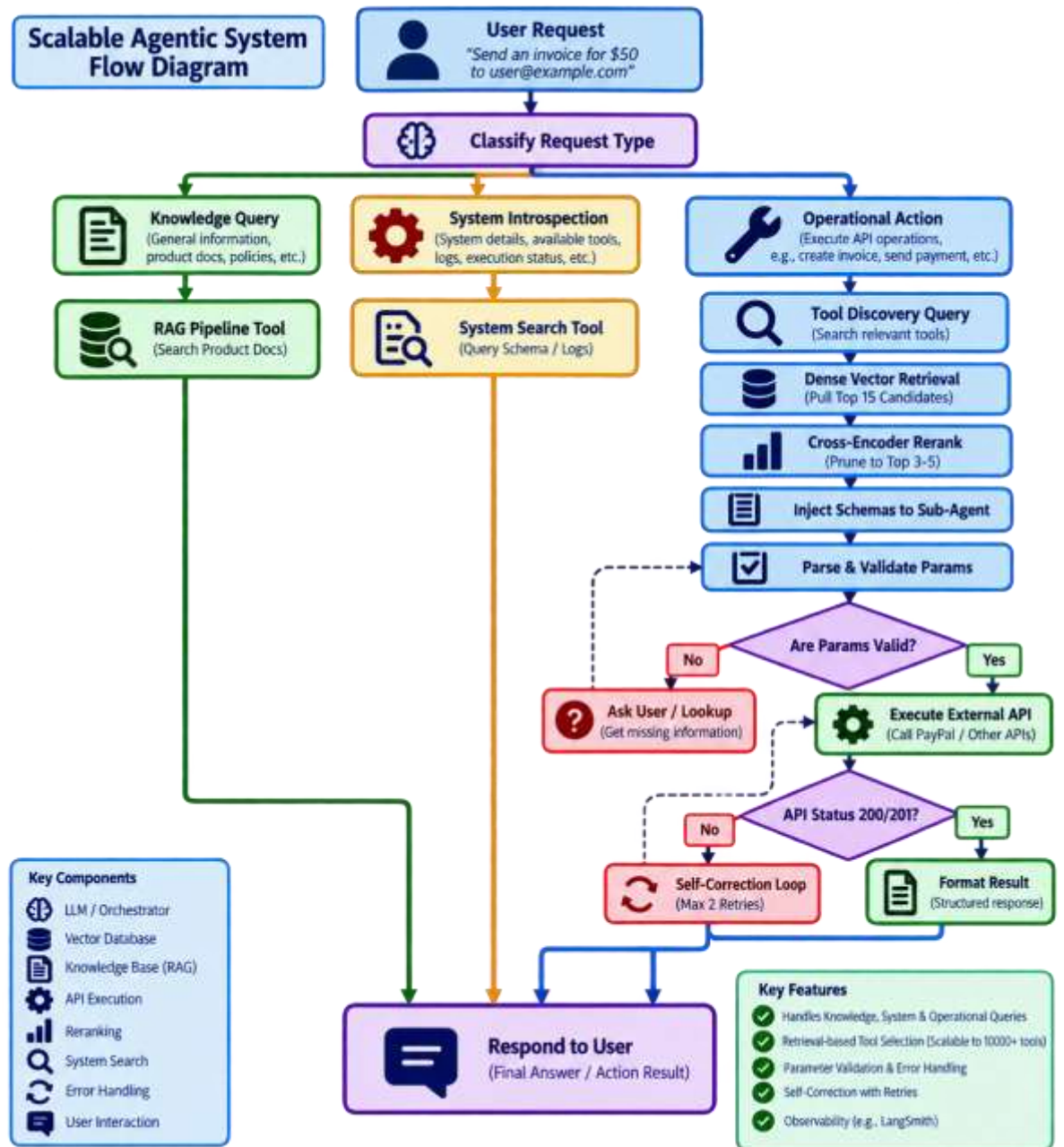
C. State Management with LangGraph

Workflows are built as state graphs with clear nodes, edges, and checkpoints:

- **Persistence:** Checkpoints are stored in PostgreSQL, allowing multi-turn conversations without state drift.
- **Resilience:** If an API call fails with a validation error, the graph routes the error back to the sub-agent for automatic self-correction instead of dropping the workflow.
- **Human-In-The-Loop (HITL):** Financial transactions above a specific threshold trigger graph interrupts that pause execution until human confirmation is granted.

End-to-End Execution Flowchart

- The following diagram illustrates how user intent flows through classification, retrieval, execution, and error handling:



Concrete Real-World Scenario: PayPal Integration

User Query:

"Send an invoice for \$50 to user@example.com"

Step-by-Step System Execution:

1. Global Intent Parsing

- The Primary Orchestrator receives the input.

- Intent: Action: Billing/Invoicing.
- Entities identified: amount: 50, currency: USD (default), recipient: user@example.com.

2. Discovery & Schema Retrieval

- The Tool Discovery Engine searches the registry using: "create send invoice to recipient".
- Top retrieved schemas:
 1. paypal_invoices_create_draft
 2. paypal_invoices_send
 3. paypal_invoices_get_details

3. Sub-Agent Plan Formulation

The Invoicing Sub-Agent identifies that sending an invoice requires a two-step sequence:

1. create_draft_invoice (creates the invoice object and returns an invoice_id).
2. send_invoice (takes invoice_id and delivers it via email).

4. Step 1: Draft Creation

- Sub-agent generates the structured JSON payload for paypal_invoices_create_draft:

```
{
  "detail": {
    "currency_code": "USD"
  },
  "primary_recipients": [
    {
      "billing_info": {
        "email_address": "user@example.com"
      }
    }
  ],
  "items": [
    {
      "name": "Standard Services",
      "quantity": "1",
      "unit_amount": {
        "currency_code": "USD",
        "value": "50.00"
      }
    }
  ]
}
```

}

- **API Call:** POST /v2/invoicing/invoices
- **Response Received:** HTTP 201 Created with {"id": "INV2-R4T7-9LK2-B3ZZ"}.

5. Step 2: Dispatch Invoice

- The Sub-Agent extracts the ID INV2-R4T7-9LK2-B3ZZ.
- Sub-agent generates the payload for paypal_invoices_send:

```
{  
  "invoice_id": "INV2-R4T7-9LK2-B3ZZ",  
  "notify_customer": true  
}
```

- **API Call:** POST /v2/invoicing/invoices/INV2-R4T7-9LK2-B3ZZ/send
- **Response Received:** HTTP 202 Accepted.

6. Final User Response

"Invoice #INV2-R4T7-9LK2-B3ZZ for \$50.00 has been created and sent to user@example.com."

Implementation of the Required Additional Tools

1. RAG Pipeline Tool (External Knowledge)

- **Purpose:** Provides technical documentation, API specifications, and policy rules without executing state-changing API operations.
- **Mechanism:**
 - Documents (PayPal Developer Documentation, fee schedules, dispute policies) are chunked and stored in a vector index.
 - When a user asks: *"What is the dispute window for chargebacks?"*, the orchestrator invokes the rag_pipeline_tool.
 - It retrieves top-3 text chunks and answers the question with citations.

2. System Search Tool (Internal Introspection)

- **Purpose:** Inspects agent capabilities, available endpoints, and transactional history.
- **Mechanism:**
 - Has direct read-only access to the internal Tool Registry and LangGraph's PostgreSQL Checkpointer.

- When a user asks: *"What tools can issue refunds?"*, the tool queries the registry metadata and returns:

"Available tools: paypal_refund_capture (refunds captured payments) and paypal_disputes_provide_refund (resolves open disputes via refund)."

- When a user asks: *"What was the status of my last request?"*, it checks the user's execution run ID in the checkpoint store and reports execution status.

Layer	Selected Framework	Rationale	Trade-offs & Alternatives Considered
Agent Orchestration	LangGraph	Provides explicit state graphs, cyclic loops (essential for API retries), and native human approval hooks.	CrewAI / AutoGen: CrewAI is conversational and role-centric, making deterministic state transitions and validation recovery difficult to control strictly.
Prompt & Routing Tuning	DSPy	Compiles optimal system prompts and few-shot routing examples systematically using metric-driven optimization.	Traditional prompt engineering relies on fragile manual prompt adjustments.
Tool Index Store	Qdrant (Hybrid Search)	Supports combined dense vector retrieval and sparse keyword search (BM25) with sub-10ms response times.	Pure vector stores without sparse search struggle with exact matching on programmatic API names.
Observability	LangSmith	Offers granular visualization of the multi-agent graph, logging tool selection accuracy, latency, and payloads.	Open-source alternatives (e.g., Arize Phoenix) require self-hosting infrastructure.

Scaling to 1,000+ Tools: Engineering Strategy

To expand from 50 PayPal APIs to thousands across different third-party services, implement the following patterns:

1. Two-Level Hierarchical Namespaces:

- Organize endpoints using structured dot-notation:
{domain}.{service}.{resource}.{action} (e.g., fintech.paypal.invoices.create, crm.hubspot.contacts.update).
- The top-level orchestrator identifies the domain/service first, pruning the active tool catalog by over 90% before running semantic similarity search.

2. Schema Virtualization (Lazy Loading):

- Do not persist full JSON schemas in execution memory. Keep lightweight summaries (name, category, 1-line description) in the routing index.
- Full OpenAPI/Pydantic schemas are fetched dynamically from a central schema repository only after the tool has been selected.

3. Composite Macro Tools (Automated Chaining):

- Common multi-step workflows (e.g., find_user_by_email \$to\$ create_invoice \$to\$ send_invoice) are encapsulated as single programmatic macro-tools.
- This reduces LLM reasoning cycles from 3 back-and-forth passes to 1 direct execution step.

4. Automated Schema Sanitization:

- Strip unnecessary metadata fields (e.g., internal documentation tags, long HTML descriptions) from OpenAPI specs before storing them, keeping schemas compact and focused on parameters and types.

Conclusion

Scaling an agentic system to hundreds or thousands of APIs cannot rely on stuffing all tool schemas into a single prompt, which causes severe hallucinations, context dilution, and high latency.

This solution overcomes that bottleneck by:

- **Decoupling Discovery from Execution:** Using dynamic vector retrieval to inject only the top 3–5 necessary schemas at runtime, keeping token usage constant.
- **Ensuring Deterministic State Management:** Leveraging **LangGraph** for structured retries, self-correction loops, and human-in-the-loop safeguards.

- **Separating Query Types:** Routing clean boundaries between operational actions, knowledge search via the **RAG Pipeline Tool**, and internal diagnosis via the **System Search Tool**.
- **Scaling via Hierarchy:** Implementing namespaces and schema virtualization to support enterprise growth seamlessly up to 1,000+ tools.